CMPT 413/713: Natural Language Processing

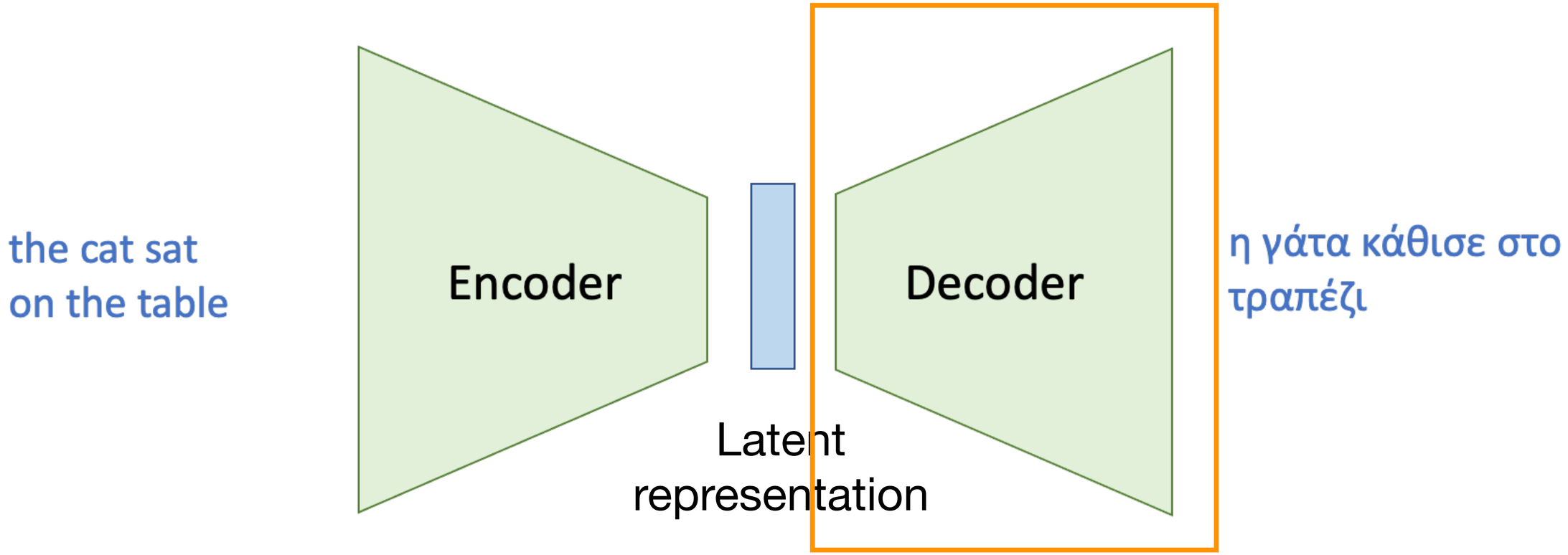
Text Generation

Fall 2021

2021-11-23

Adapted from slides from Chris Manning and Antoine Bosselut
(with some content from slides from Graham Neubig)

Encoder-Decoder Model

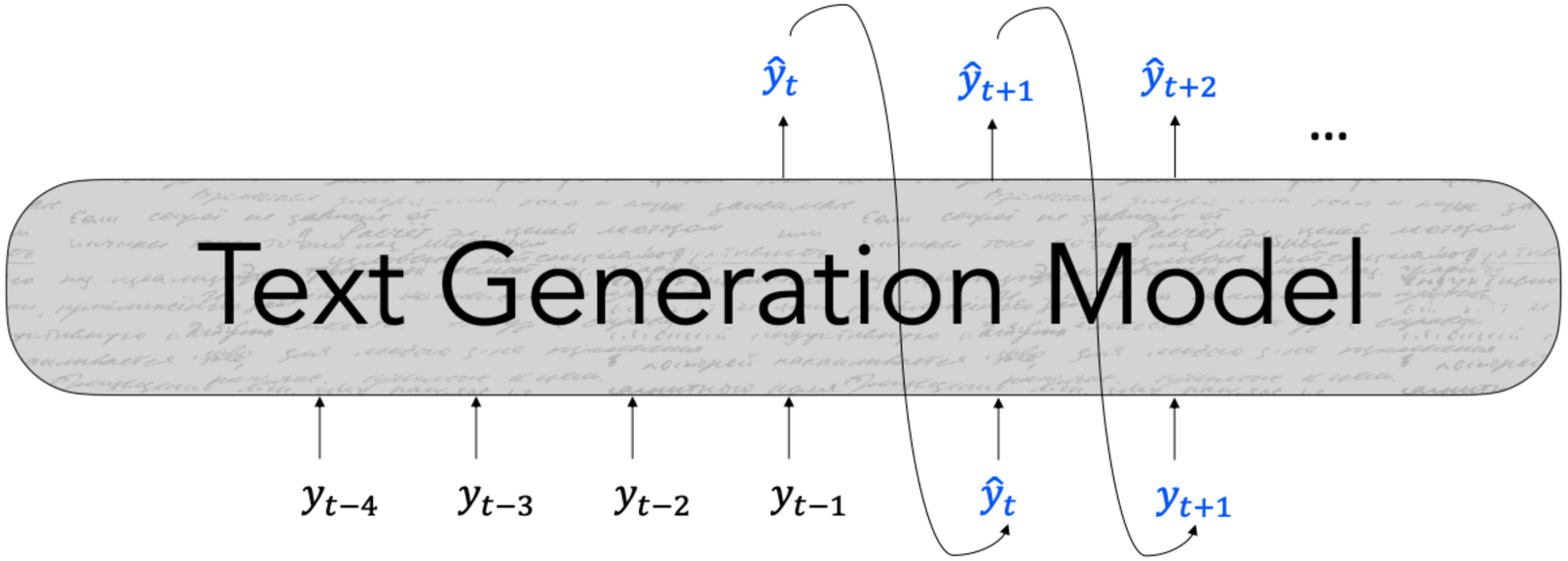


Understanding what is said
(encoding, parsing, feature extraction)

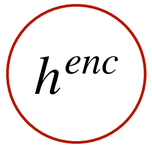
Deciding what to say
(decoding, generating)

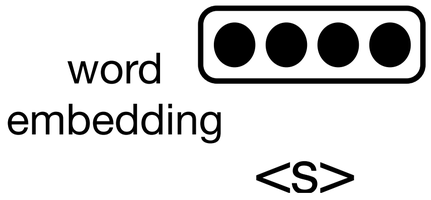
Autoregressive text generation

- Generate future words based on past words
- At each time step t, the model predicts the next token to generate ($\hat{y}_t$) based on the sequence of tokens before: $\hat{y}_t = g(\{y_{<t}\})$

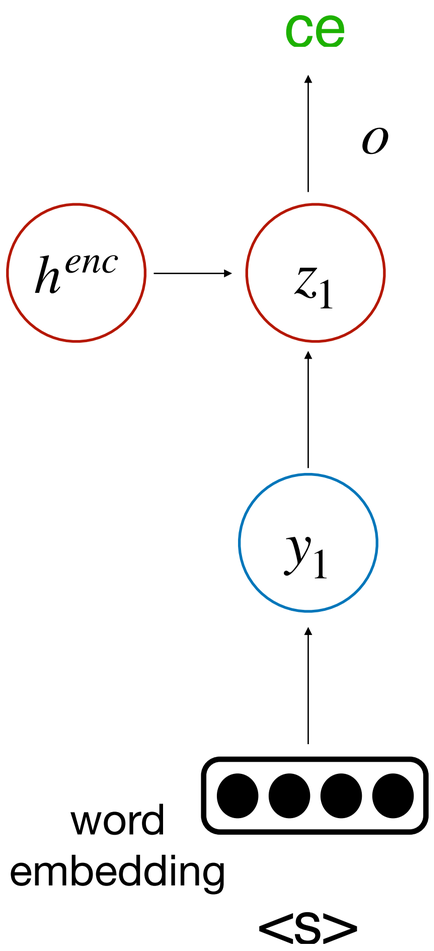


RNN Decoder

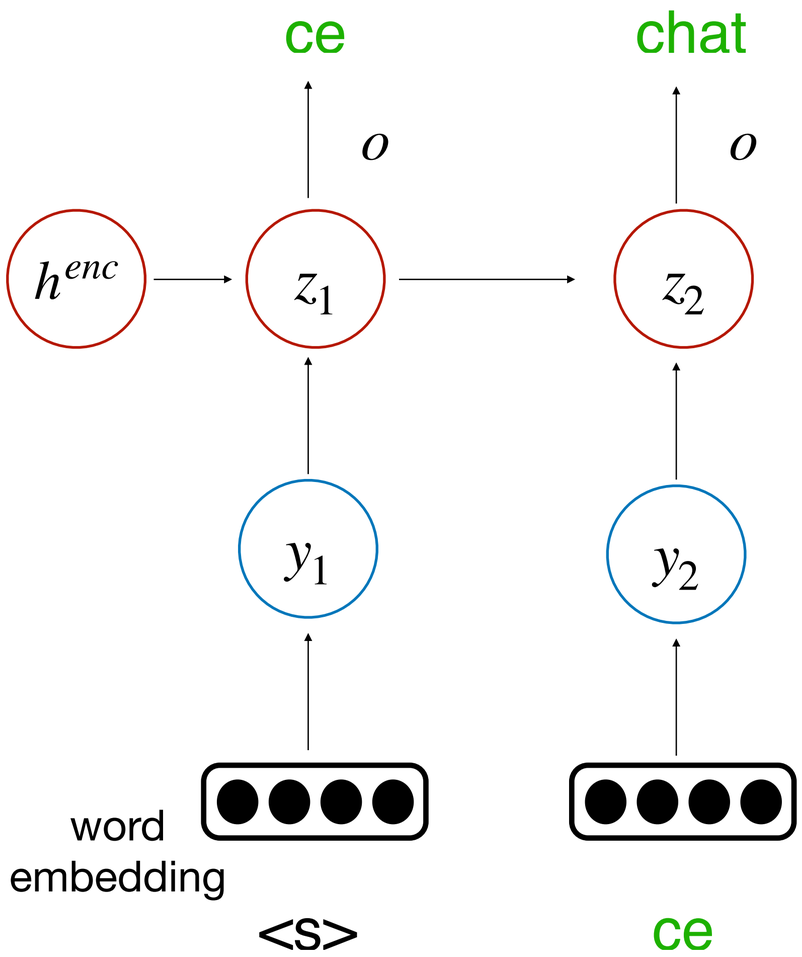




RNN Decoder

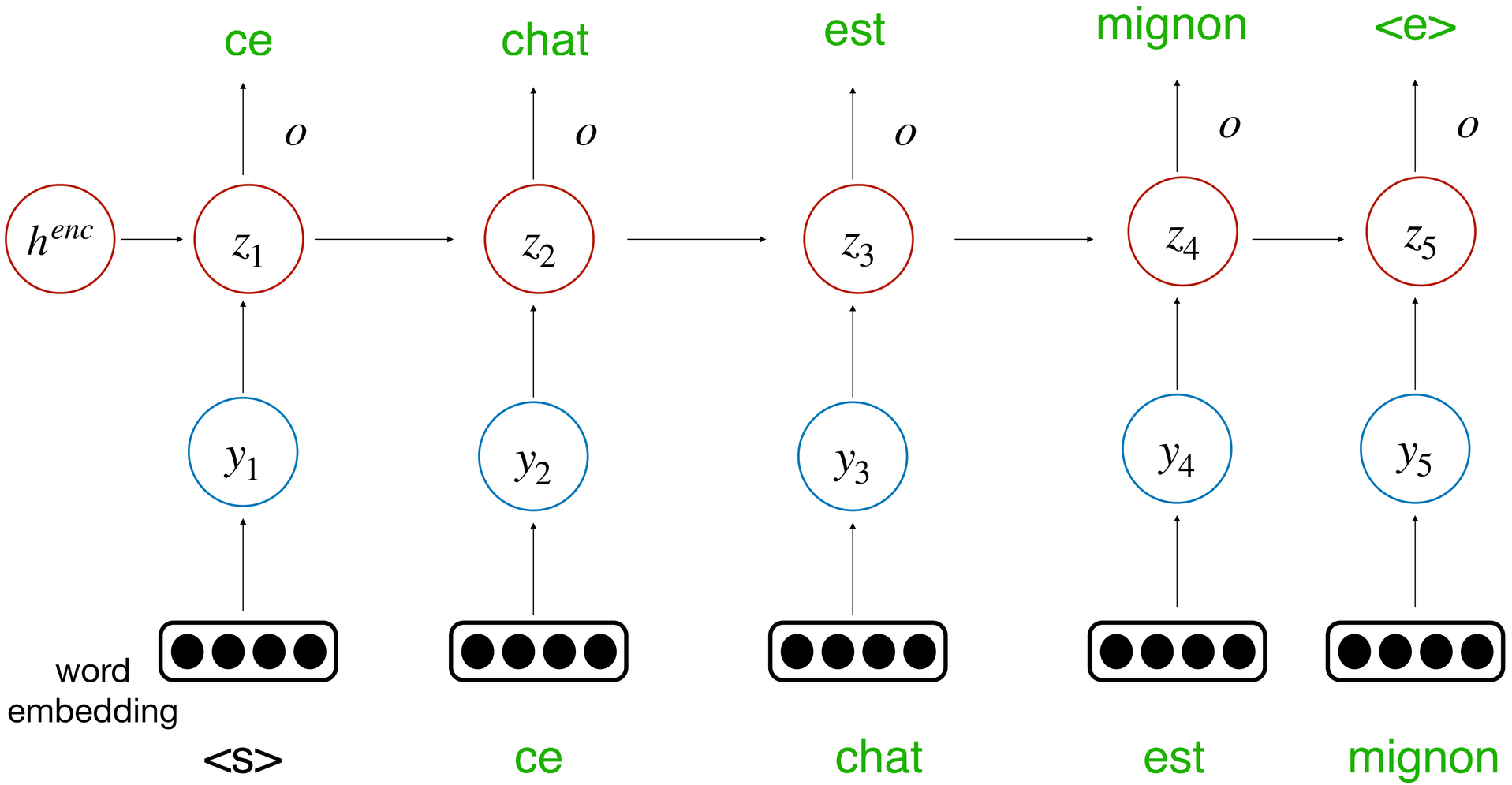


RNN Decoder



RNN Decoder

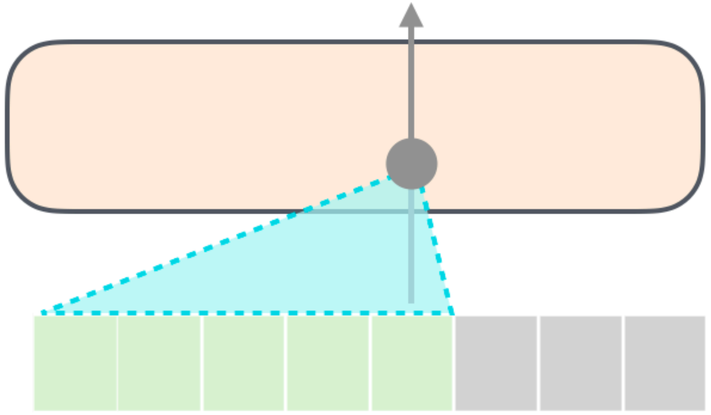
- A conditioned language model



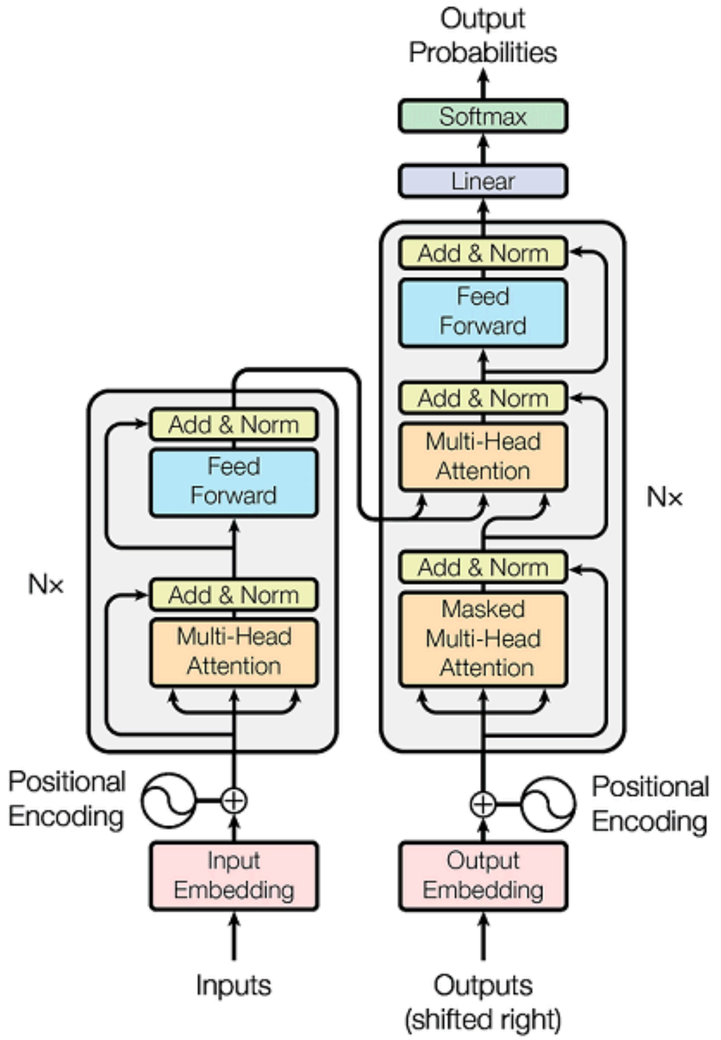
Autoregressive generation with transformers

- Encoder-Decoder Attention, where queries come from previous decoder layer and keys and values come from output of encoder

- Masked decoder self-attention on previously generated outputs



(figure credit: Jay Alammar http://jalammar.github.io/illustrated-gpt2/)



How to predict the next word?

- At each time step t, our model f computes a vector of scores for each token in our vocabulary: $S \in \mathbb{R}^{|V|}$

$$S = f(\{y_{<t}\})$$

- Then, we compute a probability distribution P_t over these scores (usually with a softmax function):

$$P_t(y_t = w|\{y_{<t}\}) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

- Our decoding algorithm then defines a function g to select a token from this distribution:

$$\hat{y}_t = g(P_t(y_t|\{y_{<t}\}))$$

Word prediction

Handling large vocabularies

- Softmax can be expensive for large vocabularies

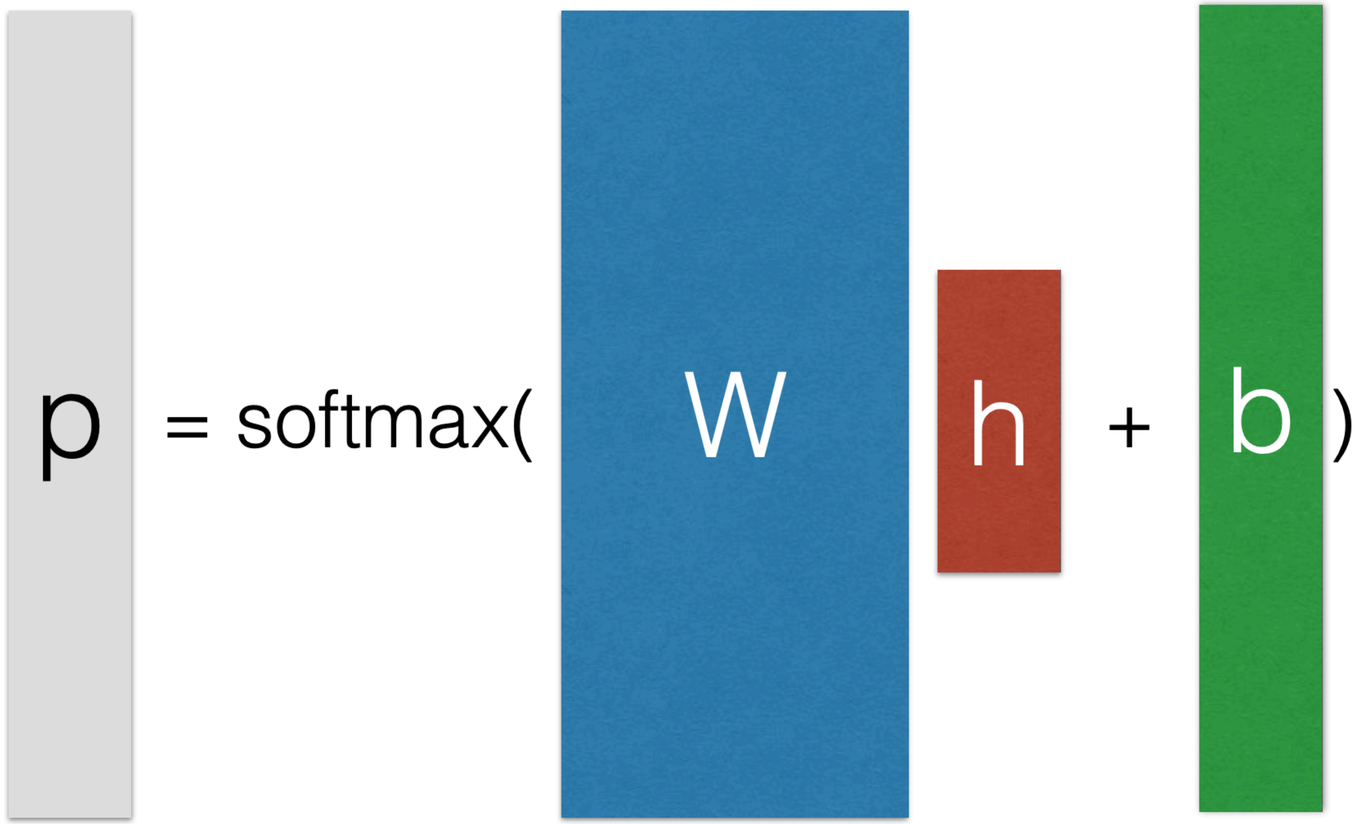
$$P(y_i) = \frac{\exp(w_i \cdot h + b_i)}{\sum_{j=1}^{|V|} \exp(w_j \cdot h + b_j)}$$

Expensive to compute

- English vocabulary size: 10K to 100K

Negative Sampling

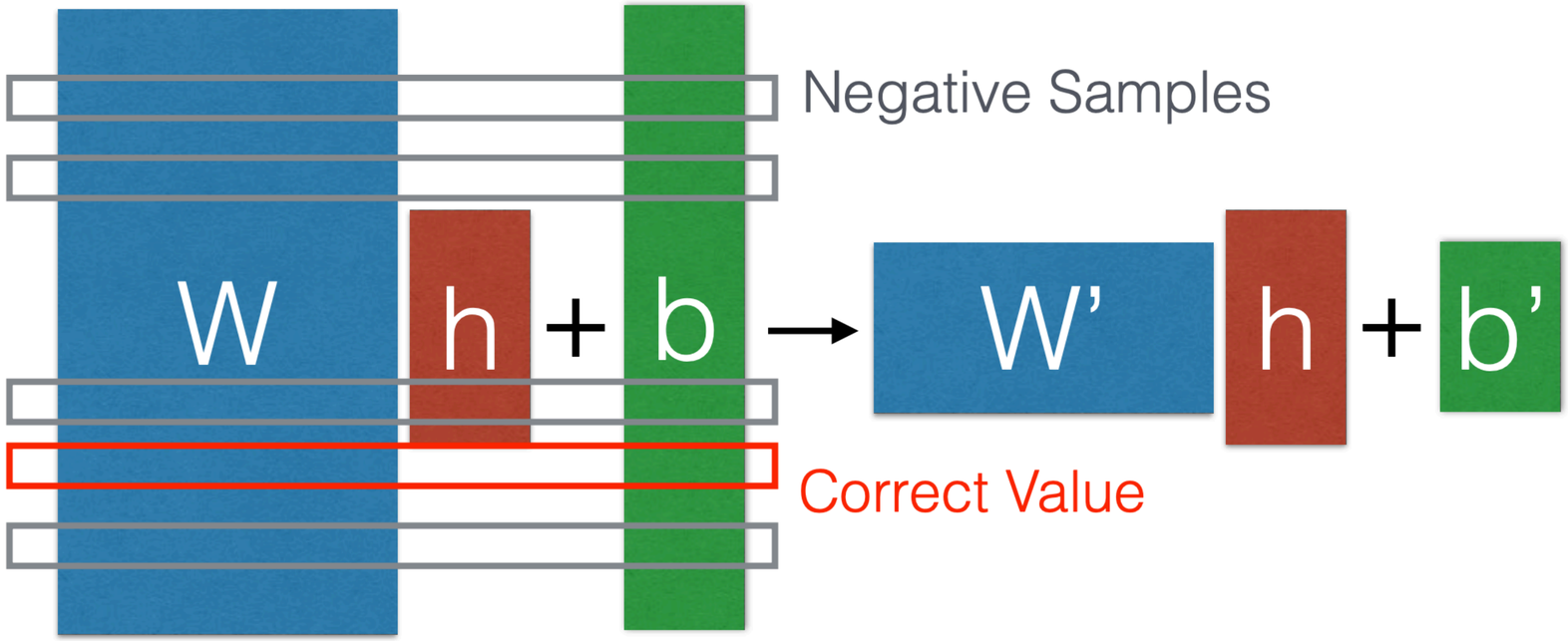
- Softmax is expensive when vocabulary size is large



(figure credit: Graham Neubig)

Negative Sampling

- Sample just a subset of the vocabulary for negative
- Saw simple negative sampling in word2vec (Mikolov 2013)



Other ways to sample:

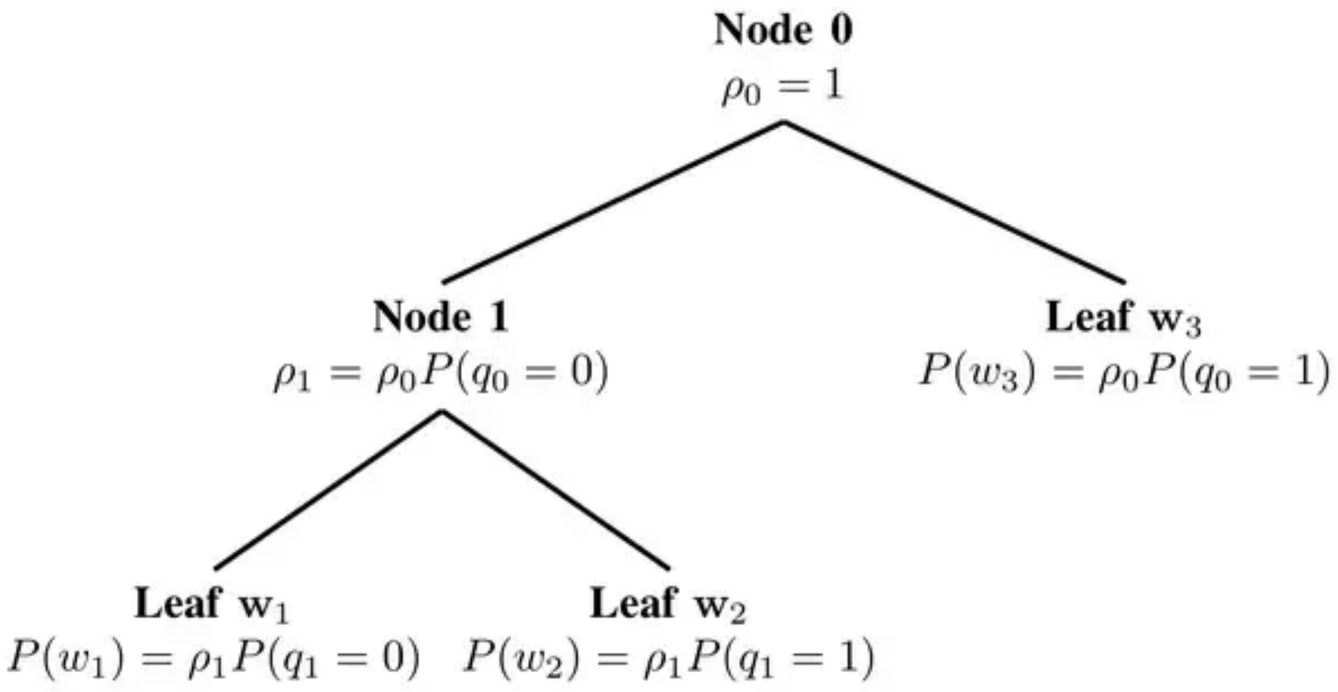
Importance Sampling (Bengio and Senecal 2003)

Noise Contrastive Estimation (Mnih & Teh 2012)

(figure credit: Graham Neubig)

Hierarchical softmax

(Morin and Bengio 2005)



(figure credit: Quora)

Class based softmax

- Two-layer: cluster words into classes, predict class and then predict word.

$$P(c|h) = \text{softmax}(W_c\, h + b_c)$$

$$P(x|c,h) = \text{softmax}(W_x\, h + b_x)$$

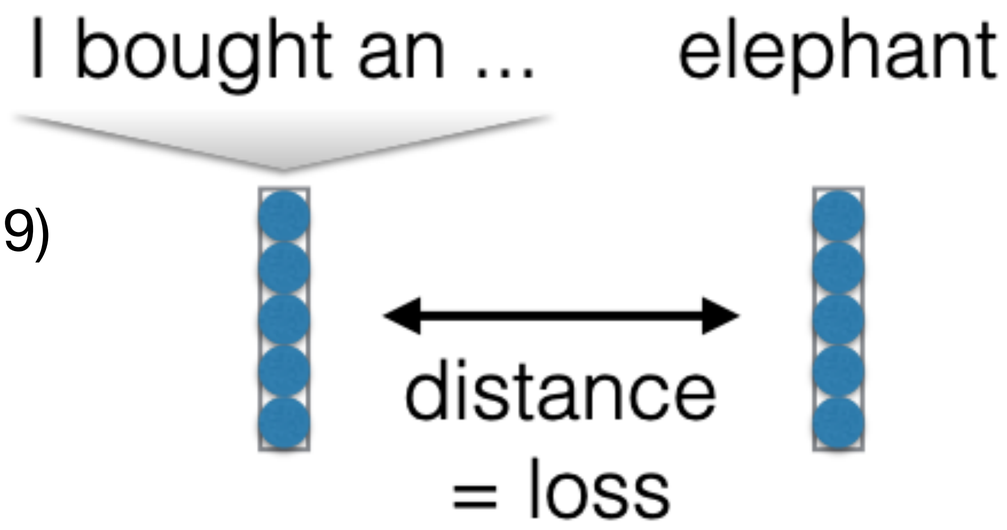
(figure credit: Graham Neubig)

- Clusters can be based on *frequency*, *random*, or *word contexts*.

(Gooding 2001, Mikolov et al 2011)

Embedding prediction

- Directly predict embeddings of outputs themselves
- What loss to use? (Kumar and Tsvetkov 2019)
  - L2? Cosine?
  - Von-Mises Fisher distribution loss, make embeddings close on the unit ball



(slide credit: Graham Neubig)

Challenges in text generation

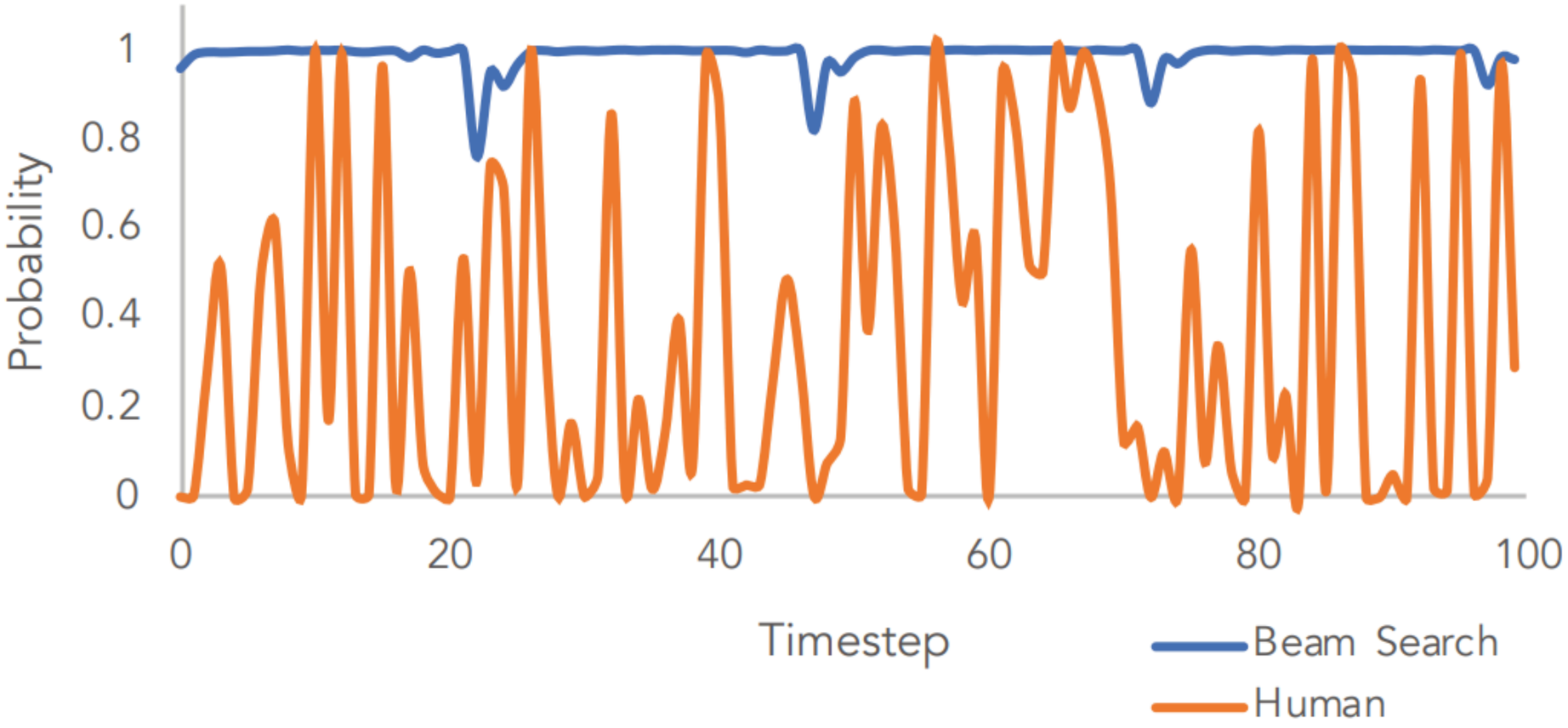
- Diversity
- Exposure Bias
- Evaluation

Repetitiveness

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Continuation: The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the **Universidad Nacional Autónoma de México (UNAM)** and **the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México...**

Human generation has lots of diversity!



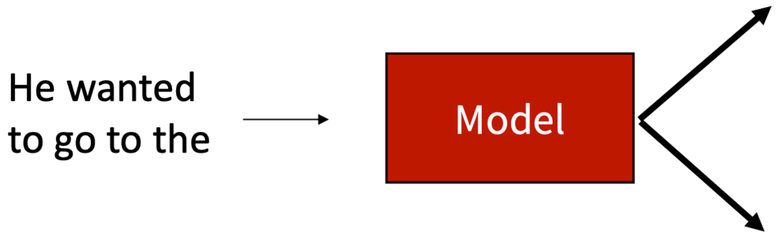
The Curious Case of Neural Text Degeneration
https://openreview.net/pdf?id=rygGQyrFvH
[Holtzman et al, ICLR 2020]

Different ways to sample during decoding

- Basic/vanilla sampling over entire distribution
- Top-k sampling
- Top-p (nucleus) sampling
- Temperature based sampling

Issues with vanilla sampling

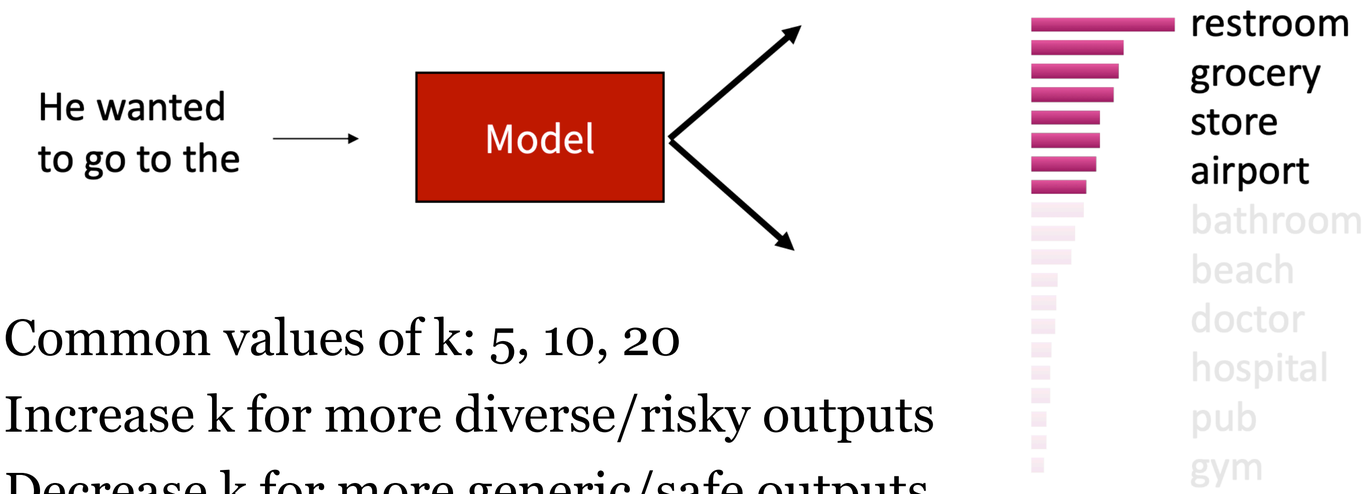
- Sample from entire probability distribution



- Long tail could have enough mass unlikely words are still selected

Decoding: Top-k sampling

- Only sample from top k tokens in the probability distribution



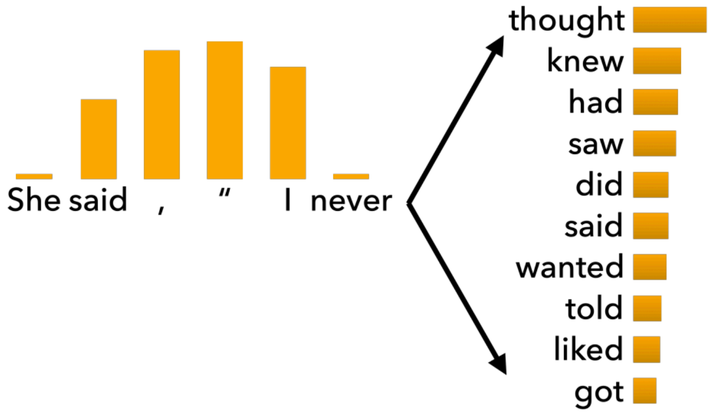
- Common values of k: 5, 10, 20
- Increase k for more diverse/risky outputs
- Decrease k for more generic/safe outputs

- Greedy search: $k = 1$, Pure sampling: $k = |V|$

(Adapted from slide by Antoine Bosselut)

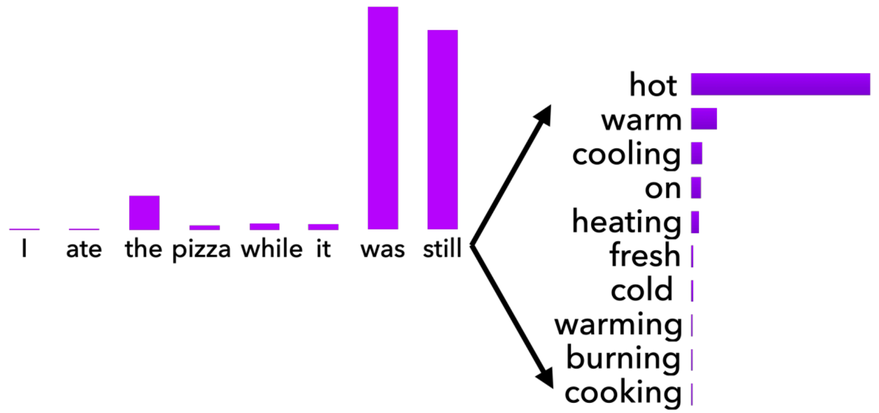
Decoding: Top-k sampling

Cuts off too slowly!



Flat distribution

Cuts off too quickly!



Peaky distribution

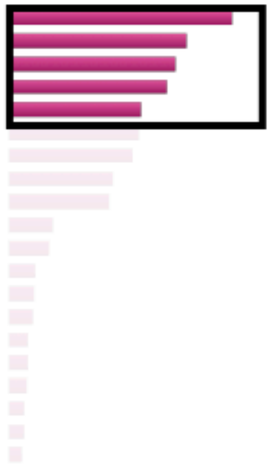
The Curious Case of Neural Text Degeneration
https://openreview.net/pdf?id=rygGQyrFvH
[Holtzman et al, ICLR 2020]

(Adapted from slide by Antoine Bosselut)

Decoding: Top-p (nucleus) sampling

- Sample from all tokens in the top p cumulative probability mass
- This allows k to vary depending on the peakiness of the distribution P_t

$P_t^1(y_t = w \mid \{y\}_{<t})$



$P_t^2(y_t = w \mid \{y\}_{<t})$

$P_t^3(y_t = w \mid \{y\}_{<t})$

(Adapted from slide by Antoine Bosselut)

Improving decoding: Temperature scaled softmax

- Recall: On timestep t, the model samples from the distribution P_t which is computed by taking the softmax of a vector of scores $S \in \mathbb{R}^{|V|}$

$$P_t(y_t = w) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

- We can apply a temperature hyperparameter τ to the softmax to rebalance the distribution

$$P_t(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{w' \in V} \exp(S_{w'}/\tau)}$$

- Raise the temperature $\tau > 1$: P_t becomes more uniform
  - More diverse output (probability is spread around vocabulary)
- Lower the temperature $\tau < 1$: P_t becomes more spiky
  - Less diverse output (probability is concentrated on top words)

Note: temperature scaled softmax is not a decoding algorithm!
It's a technique you can apply at test time, in conjunction with a decoding algorithm (such as beam search or sampling)

Improving Decoding: Re-ranking

- Decode a bunch of sequences and re-rank with a score that measure the quality of the sequences
- Have a separate scoring function to approximate the quality of the sequences
  - Simplest is to use perplexity
  - Re-rankers can score a variety of properties
    - style, logical consistency, factuality, etc
  - Can combine these different rankers (but beware of poorly-calibrated re-rankers)

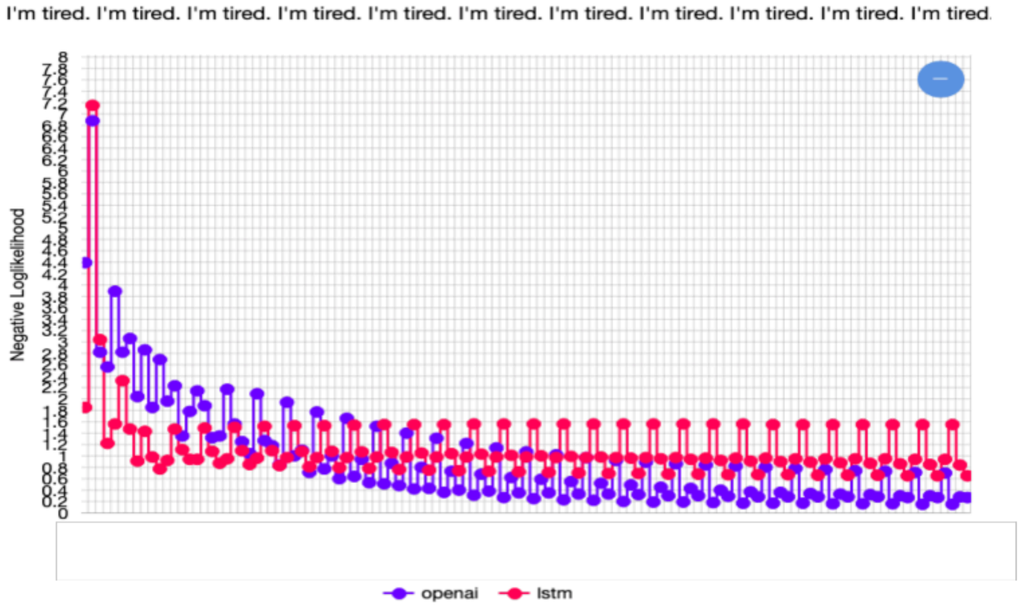
Training the model

- Maximum Likelihood Training: trained to minimize the negative log-likelihood of the next token y_t^* given the preceding tokens in the sequence $\{y^*\}_{<t}$

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(y_t^* | \{y_{<t}^*\})$$

- Maximum Likelihood Training discourages diverse text generation.

Also known as teacher forcing



Unlikelihood Training

- Given a set of undesired tokens $\mathcal{C}$, lower their likelihood in context

$$\mathcal{L}_{UL}^{t} = -\sum_{y_{neg} \in \mathcal{C}} \log(1 - P(y_{neg} \mid \{y^*\}_{<t}))$$

- Keep *teacher forcing* objective and combine them for final loss function

$$\mathcal{L}_{MLE}^{t} = -\log P(y_t^* | \{y^*\}_{<t}) \qquad \mathcal{L}_{ULE}^{t} = \mathcal{L}_{MLE}^{t} + \alpha \mathcal{L}_{UL}^{t}$$

- Set $\mathcal{C} = \{y^*\}_{<t}$ and you'll train the model to lower the likelihood of previously-seen tokens!
  - Limits repetition!
  - Increases the diversity of the text you learn to generate!

Neural text generation with unlikelihood training
https://openreview.net/pdf?id=SJeYe0NtvH
[Welleck et al, ICLR 2020]

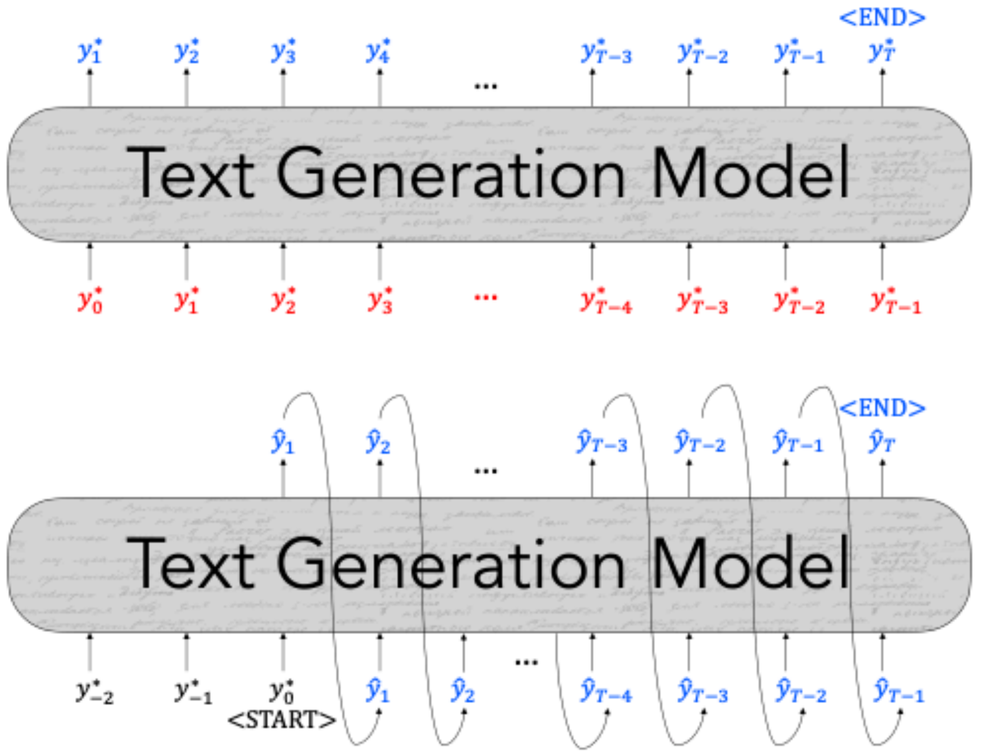
Exposure bias

- Discrepancy in model input between training and generation time
- During training, model inputs are gold context tokens

$$\mathcal{L}_{MLE} = -\sum_{t=1}^{T} \log P(y_t^*|\{y_{<t}^*\})$$

- At generation time, inputs are previously-decoded tokens

$$\mathcal{L}_{dec} = -\sum_{t=1}^{T} \log P(\hat{y}_t|\{\hat{y}_{<t}\})$$



Student forcing: use predicted tokens during training

Scheduled sampling: use decoded token with some probability p, increase p over time

Use reinforcement learning

- Sample sequence from your model
- Increase probability of sampled token in the same context
  - Proportional to reward based on reward function

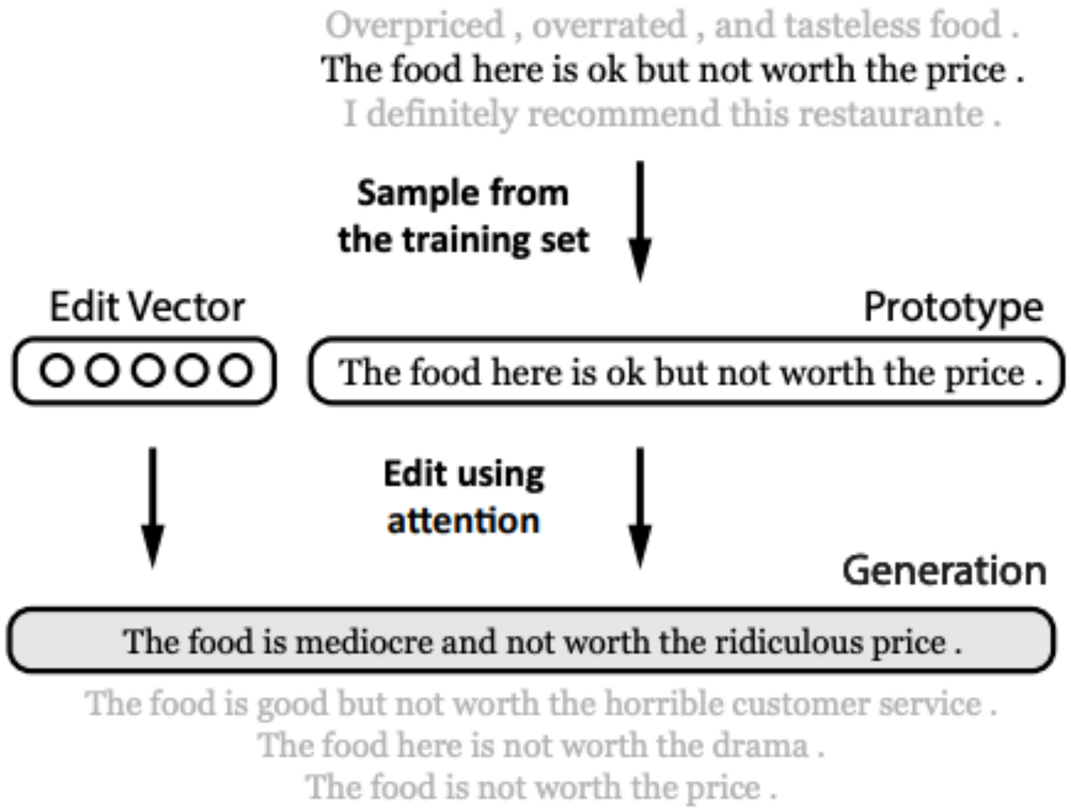
$$\mathcal{L}_{RL} = -\sum_{t=1}^{T} r(\hat{y}_t) \log P(\hat{y}_t | \{y^*\}; \{\hat{y}_{<t}\})$$

  - Reward is based on evaluation metric (BLEU, ROUGE, etc)
- Be careful about "gaming" the reward
  - Your metric will go up! But will your generation actually be better?

"even though RL refinement can achieve better BLEU scores, it barely improves the human impression of the translation quality" – Wu et al., 2016

Retrieve and Edit

- Retrieve prototype sentence x’ from a corpus
- Sample edit vector z (encodes type of edit to be perform).
- Use neural editor to combine edit vector z and prototype sentence x’ to get new sentence x.



Generating Sentences by Editing Prototypes
https://arxiv.org/pdf/1709.08878.pdf
Guu et al, TACL 2020]

Non-autoregressive generation (with transformers)

- Can generate words in a non-autogressive manner
- Relies on the idea of masked language model
- Predict length of output
- Iterative refinements / masking
  - Predict length of output
  - Predict all words $P(y_i|x)$ $\hat{y}_t^0 = \arg\max_{y_t} \log p(y_t^0|X)$
  - Iteratively refine sequence of predictions based on input and previous predictions

$$\hat{y}_t^l = \arg\max_{y_t} \log p(y_t^l|\hat{Y}^{l-1}, X)$$

- Efficient decoding since parts of the decoding can run in parallel

Each iteration, can just mask out low-confidence words

Mask-Predict: Parallel Decoding of Conditional Masked Language Models
https://arxiv.org/pdf/1904.09324.pdf
[Ghazvininejad et al, EMNLP 2019]

Evaluation

Content Overlap Metrics

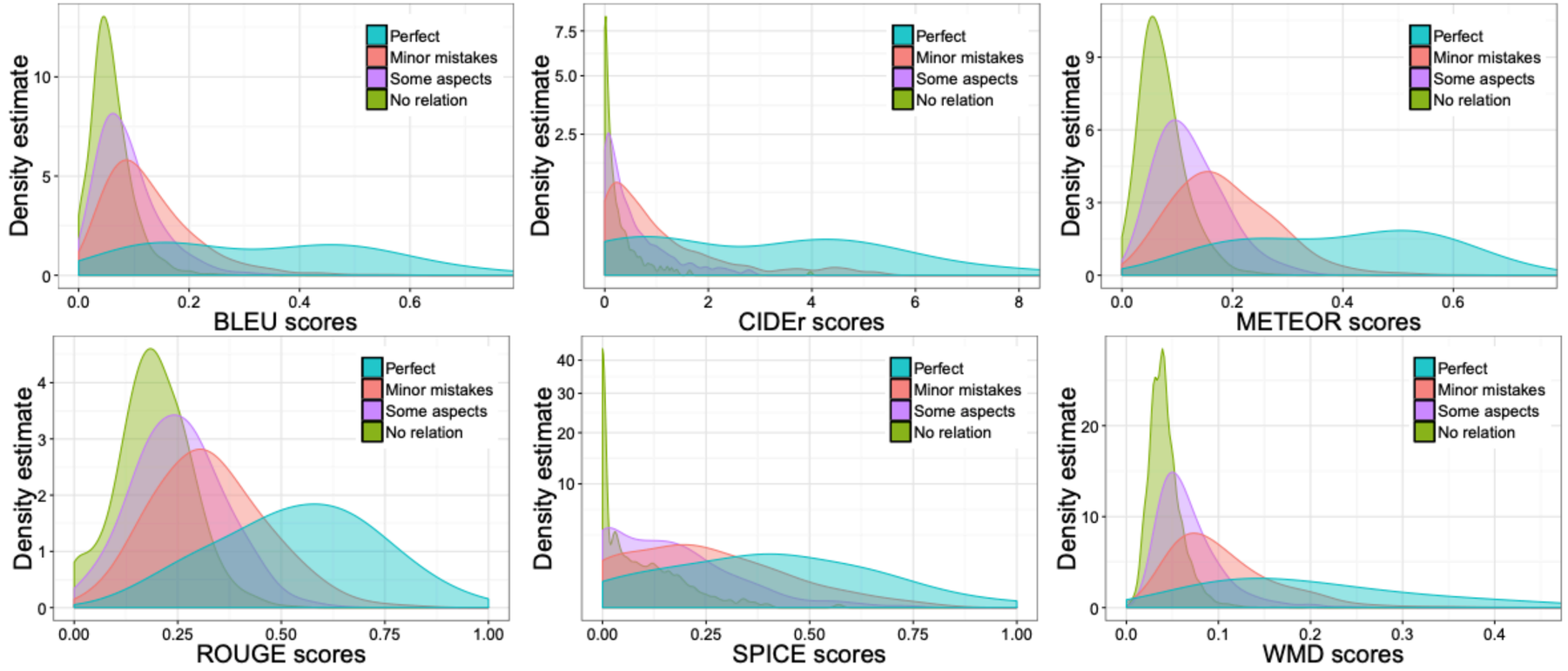
Model-based Metrics

Human Evaluations

Content overlap metrics

- Compute a score that indicates the similarity between *generated* and *gold-standard (human-written) text*
- Fast and efficient and widely used
- Two broad categories:
  - *N*-gram overlap metrics (e.g., **BLEU**, ROUGE, METEOR, CIDEr, etc.)
  - Semantic overlap metrics (e.g., PYRAMID, SPICE, SPIDEr, etc.)

	FLICKR-8K			COMPOSITE		
	Pearson	Spearman	Kendall	Pearson	Spearman	Kendall
WMD	0.68	0.60	0.48	**0.43**	0.43	0.32
SPICE	**0.69**	**0.64**	**0.56**	0.40	0.42	**0.34**
CIDEr	0.60	0.56	0.45	0.32	0.42	0.32
METEOR	**0.69**	0.58	0.47	0.37	**0.44**	0.33
BLEU	0.59	0.44	0.35	0.34	0.38	0.28
ROUGE	0.57	0.44	0.35	0.40	0.39	0.29



Re-evaluating Automatic Metrics for Image Captioning
[Kilickaya et al, EACL 2017]

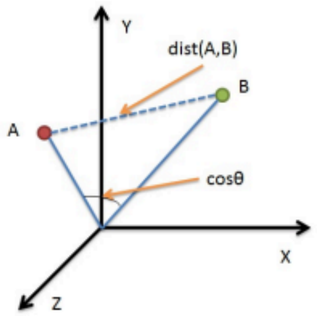
N-gram overlaps are not good metrics

- Not ideal for machine translation
- But they get even progressively worse for tasks that are more open-ended than machine translation
  - Worse for summarization, as longer output texts are harder to measure
  - Much worse for dialogue, which is more open-ended that summarization
  - Much, much worse story generation, which is also open-ended, but whose sequence length can make it seem you're getting decent scores!

Model-based metrics

- Use learned representations of words and sentences to compute semantic similarity between generated and reference texts
- No more n-gram bottleneck because text units are represented as embeddings!
- Even though embeddings are pretrained, distance metrics used to measure the similarity can be fixed

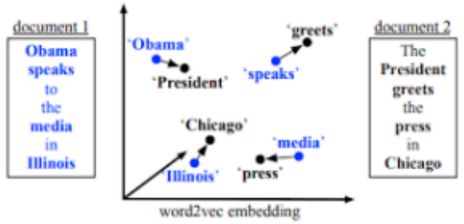
Model-based metrics: Word distance functions



Vector Similarity:

Embedding based similarity for semantic distance between text.

- **Embedding Average** (Liu et al., 2016)
- **Vector Extrema** (Liu et al., 2016)
- **MEANT** (Lo, 2017)
- **YISI** (Lo, 2019)

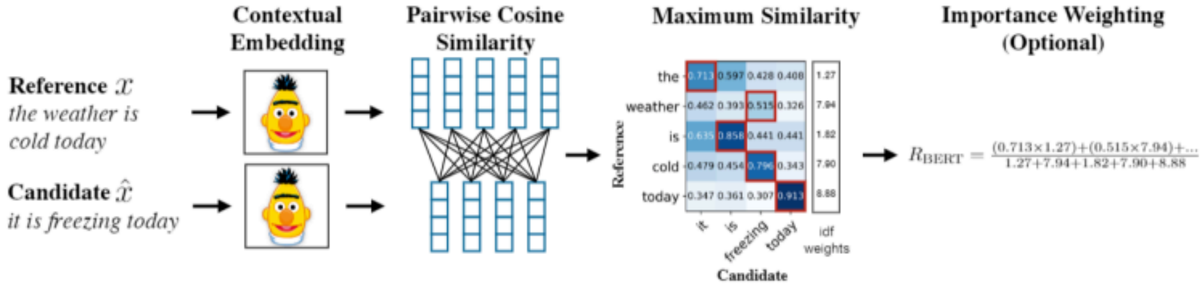


Word Mover's Distance:

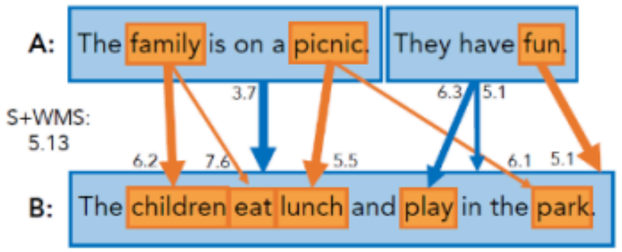
Measures the distance between two sequences (e.g., sentences, paragraphs, etc.), using word embedding similarity matching.
(Kusner et.al., 2015; Zhao et al., 2019)

BERTSCORE:

Uses pre-trained contextual embeddings from BERT and matches words in candidate and reference sentences by cosine similarity.
(Zhang et.al. 2020)



Model-based metrics: Beyond word matching



Sentence Movers Similarity :

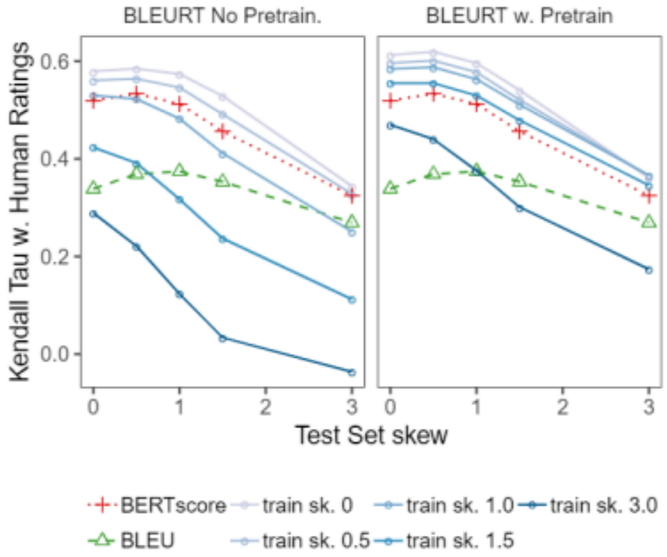
Based on Word Movers Distance to evaluate text in a continuous space using sentence embeddings from recurrent neural network representations.

(Clark et.al., 2019)

BLEURT:

A regression model based on BERT returns a score that indicates to what extend the candidate text is grammatical and conveys the meaning of the reference text.

(Sellam et.al. 2020)



Human evaluation

- Ask humans to evaluate the quality of generated text
- Overall or along some specific dimension: fluency, coherence / consistency, factuality and correctness, commonsense, style / formality, grammaticality, typicality, redundancy
- Drawbacks
  - Cannot compare across studies
  - Inconsistent
  - Expensive

Seq2Seq Tasks and Applications

Task/Application	Input	Output
Machine Translation	French	English
Summarization	Document	Short Summary
Dialogue	Utterance	Response
Parsing	Sentence	Parse tree (as sequence)
Question Answering	Context + Question	Answer